Assignment 5

Technical Analysis using R, Development Phase

Student: May Elmady

Course: BDA400 – Data Science Tools and Techniques

College: CDI College

Date: November 2026

Repository: https://github.com/may0elmady-ship-it/bda400-technical-analysis-may-elmady

May Elmady
BDA400 – Data Science Tools and Techniques
Assignment 5 – Technical Analysis using R, Development Phase

1. Overview

Each technical indicator is implemented in its own R script, following the function names and arguments of the course templates exactly so the instructor's test file can call them. Only base R core functions are used (for loops, sum, length, sqrt, abs, seq); no packages or library indicator functions. A test script runs every example from the brief and 20 automated checks, and a demo script shows the indicators working on a stock data frame with simple buy/sell rules. To make the work my own, every indicator is also tested on real-style data from my business, Toronto Cakes, a custom cake shop I run through Instagram (40,000+ followers): weekly orders, butter prices from my supplier, follower growth, servings per cake, monthly revenue, average cake price, and wedding vs birthday orders. The same tools traders use on stock prices help me read my own business trends.

2. Simple Moving Average – sma.R

sma(data, period) stops with an error if the data is shorter than the period. It then slides a window of 'period' values across the data, adds them with sum() and divides by the period. The result has length n – period + 1. My example: my weekly Instagram orders over 9 weeks were 14, 17, 15, 21, 19, 24, 28, 26, 23. A 3-week SMA gives 15.33, 17.67, 18.33, 21.33, 23.67, 26.00, 25.67, which hides the slow weeks and shows a clear upward trend in demand.

3. Exponential Moving Average – ema.R

ema(data, period) computes the multiplier 2 / (period + 1). The first EMA equals the first price; every later value is (price – previous EMA) × multiplier + previous EMA. This gives more weight to recent prices. The output is the same length as the input. My example: the butter price per kg I paid my supplier (9.80 to 11.90 CAD over 9 purchases) gives an EMA(3) that climbs from 9.80 to 11.60, reacting faster to recent price jumps than an SMA, which helps me decide when to adjust cake prices.

4. MACD – macd.R

macd(data, short_period, long_period, signal_period) reuses ema(): MACD line = short EMA – long EMA, signal line = EMA of the MACD line over signal_period, histogram = MACD line – signal line. It returns a named list (macd_line, signal_line, histogram). The script loads ema.R automatically if it is not already loaded. My example: my follower count (thousands) during a wedding-season promo, 40.0 to 42.1, gives a MACD line that stays positive after day 1, meaning the promotion created real upward momentum.

5. Standard Deviation – stdev.R

stdev(data) follows the population formula from the brief: compute the mean (sum / n), build the differences from the mean with a loop, square them, average the squares to get the variance, then take sqrt(). My example: servings per cake for 9 orders (12, 20, 15, 30, 24, 40, 18, 60, 25) give a standard deviation of 14.06, showing my order sizes vary a lot between small birthday cakes and large wedding cakes, so I must plan ingredients and oven time per order.

6. Linear Regression – linreg.R

linreg(regressionSource, regressionLength, regressionOffset) validates that the length is not larger than the data and that the offset is smaller than the length. It selects the window using start = max(1, n – length + offset) and end = min(n, n – offset), exactly as in the pseudocode, numbers the points 1..k, and computes the least-squares slope = Σ(x–x■)(y–■) / Σ(x–x■)² and intercept = ■ – slope·x■. It returns slope, intercept and the predicted values of the fitted line. My example: monthly revenue (in hundreds of CAD) of 32, 35, 34, 39, 42, 45, 47, 51 with regressionLength = 6 gives a slope of about 2.86, meaning revenue grows by roughly $286 per month.

7. Relative Strength Index – rsi.R

rsi(data, period) takes the differences between consecutive prices and splits them into gains and losses (absolute value). The starting average gain and loss come from the first 'period' changes, then Wilder's smoothing updates them: avg = (avg × (period – 1) + current) / period. RSI = 100 – 100 / (1 + RS) with RS = avg gain / avg loss. The first 'period' values are NA. Two safety checks were added: if there are no losses RSI is 100 (avoids dividing by zero), and if the data is too short for the period the function warns and returns NA values instead of crashing. My example: my average price per cake over 10 weeks (85 to 104 CAD) gives RSI(5) values of 44.44, 62.48, 68.22, 72.20, 59.72. The value above 70 tells me prices rose quickly that week, so a small pullback or promotion may follow.

8. Stochastic RSI – stoch_rsi.R

stoch_rsi(data, period, k_period, d_period) calls rsi(), normalizes the RSI values to 0–1 with (RSI – min) / (max – min), then %K = sma(StochRSI, k_period) and %D = sma(%K, d_period). It returns k_line and d_line. NA values from the RSI warm-up are ignored when finding the min and max, and a flat RSI is handled to avoid dividing by zero. Note: the brief's example uses period = 14 on only 10 prices, so RSI cannot be computed and the result is NA; my test file also runs it on 12 weeks of my Toronto Cakes orders with period = 5, where %K stays between 0 and 1 as expected.

9. Crossover – crossover.R

crossover(arr1, arr2) checks the arrays have equal length, sets the first signal to "None", and for each later point returns "Up" when arr1 moves from at or below arr2 to above it, "Down" when it moves from at or above to below, and "None" otherwise, matching the pseudocode.

My example: wedding orders per month (2, 3, 4, 7, 9, 11, 12, 8, 5) vs birthday orders (9, 9, 8, 7, 8, 8, 9, 9, 10). The result is "Up" in month 5, when wedding orders overtake birthdays (wedding season starts), and "Down" in month 8 when they fall back.

10. Crossunder – crossunder.R

crossunder(arr1, arr2) checks equal length, sets the first value to "None", and returns "True" when arr1 moves from greater than or equal to arr2 to below it, otherwise "False". The output format (string values) follows the provided pseudocode so it matches the test file. My example: with the same wedding vs birthday data, crossunder returns "True" only in month 8, the point where wedding season ends and I can expect more birthday orders again.

11. Testing

test_indicators.R sources all nine scripts, runs each indicator on my Toronto Cakes data and on the brief's sample data, and performs 23 checks (hand-calculated values, error handling, edge cases such as identical orders and prices that only go up). All 23 checks pass: "Passed: 23 Failed: 0".

12. How to Run the Code

1) Download or clone the repository and open the folder in RStudio. 2) Set the working directory to that folder, for example setwd("path/to/bda400-technical-analysis"). 3) Run source("test_indicators.R") to see all results and the pass/fail summary. Each indicator file also ends with a commented Toronto Cakes example that can be uncommented and run on its own. 4) Run source("demo_stock_analysis.R") to build a stock data frame (Date, Open, High, Low, Close, Volume), add SMA, EMA, RSI and MACD columns, and generate BUY/SELL signals from SMA5/SMA10 crossovers. To use a real stock file: Rscript demo_stock_analysis.R your_stock.csv. No packages need to be installed.

Repository files

```
sma.R  ema.R  macd.R  stdev.R  linreg.R  rsi.R
stoch_rsi.R  crossover.R  crossunder.R
test_indicators.R  demo_stock_analysis.R  README.md
```